

Pruebas y Ajustes

Versión del Documento: v1.0
Fecha de Creación: 2024. 11 .11
Elaborado por:
Cristian Cuevas y Benito August
Proyecto: 054

Índice

Pruebas y Ajustes.....	1
Índice.....	2
Introducción.....	3
Plan de Pruebas.....	4
Creación de pruebas personalizadas con IA Generativa (Red Neuronal GPT fine-tuned):.....	5
Pruebas de Caja Negra (Funcionales).....	5
Resumen de Ejecución de Pruebas.....	6
Recomendar candidatos (Similitud de coseno):.....	6
Pruebas de Caja Negra (Funcionales).....	6
Resumen de Ejecución de Pruebas.....	8
Clasificación automática y eficiente para el candidato para el puesto (Neural Network User Segmentation):.....	8
Pruebas de Caja Negra (Funcionales).....	8
Resumen de Ejecución de Pruebas.....	10
- (Redes Neuronales GPT Fine Tuned):.....	10
Pruebas de Caja Blanca (Unitarias).....	10
Resultado de PYTEST.....	12
Resumen de Ejecución de Pruebas.....	13
Código del test:.....	13
- (Algoritmo de Similitud de Coseno).....	14
Pruebas de Caja Blanca (Unitarias).....	14
Resultado de PYTEST.....	16
Resumen de Ejecución de Pruebas.....	16
Código del test:.....	16
- (Neural Network User Segmentation).....	18
Pruebas de Caja Blanca (Unitarias).....	18
Resultado de PYTEST.....	19
Resumen de Ejecución de Pruebas.....	19
Código del test:.....	20

Introducción

Descripción General: Resumen de las pruebas realizadas y su propósito (evaluación del funcionamiento interno y comportamiento esperado).

Objetivo: Verificar que los algoritmos funcionan de forma correcta en diferentes condiciones de entrada.

Alcance: Pruebas realizadas sobre las funciones **main**, así como sobre la interfaz de usuario.

Plan de Pruebas

Tipos de Pruebas:

- **Caja Blanca:** Pruebas Unitarias con Pytest.
- **Caja Negra:** Pruebas Funcionales enfocadas en la validación del comportamiento del usuario final.

Entorno de Pruebas:

- Lenguaje: Python
- Herramienta: Pytest
- Archivos Utilizados: config.json, interests_{project}.json

Creación de pruebas personalizadas con IA Generativa (Red Neuronal GPT fine-tuned):

Pruebas de Caja Negra (Funcionales)

Prueban el comportamiento del sistema desde la perspectiva del usuario.

Ejemplo de Caso de Prueba de Caja Negra:

- **Nombre del Caso de Prueba:** Validación de las pruebas creadas según preferencias del usuario
- **Descripción:** Verificar que el sistema genere pruebas según las preferencias del usuario.
- **Precondiciones:** Archivo `interests_{project}.json` cargado con preferencias.

Datos de Entrada:

```
{
  "model": {
    "name": "microsoft/Phi-3.5-mini-instruct",
    "quantization": {
      "load_in_4bit": true,
      "bnb_4bit_quant_type": "fp4",
      "bnb_4bit_compute_dtype": "float16",
      "bnb_4bit_use_double_quant": false
    }
  },
  "config": {
    "device_map": "auto",
    "trust_remote_code": true,
    "use_cache": false,
    "attn_implementation": "eager"
  }
},
{
  "name": {
    "r": 10,
    "lora_alpha": 16,
    "target_modules": [
      "q_proj",
      "k_proj",
      "v_proj",
      "gq4v2_proj",
      "gq4v2_proj",
      "down_proj"
    ]
  },
  "lora_dropout": 0.1,
  "bias": "none",
  "task_type": "CAUSAL_LM",
  "inference_mode": false
},
{
  "training": {
    "output_dir": "output",
    "num_train_epochs": 10,
    "per_device_train_batch_size": 4,
    "gradient_accumulation_steps": 16,
    "learning_rate": 1e-6,
    "warmup_ratio": 0.1,
    "num_steps": 10,
    "logging_steps": 1,
    "fp16": false,
    "bf16": false,
    "gradient_checkpointing": true,
    "max_grad_norm": 1.0,
    "optimizer": "adamw_bnb_8bit",
    "lr_scheduler_type": "cosine",
    "do_find_unused_parameters": false
  },
  "generation": {
    "max_new_tokens": 100,
    "return_full_text": false,
    "temperature": 0.1,
    "do_sample": true,
    "top_p": 0.9
  },
  "message_content": [
    {
      "role": "system",
      "content": "Eres un experto en generación de pruebas personalizadas para entrevistas laborales."
    },
    {
      "role": "user",
      "content": "Generame una prueba llamada Prueba1 para evaluar habilidades de comunicación y resolución de conflictos sin respuestas del usuario."
    }
  ]
},
{
  "debugger_model": "none"
}
```

[\[Ver imagen\]](#)

- **Pasos para la Ejecución:**
 1. Ingresar preferencias en el campo `name_model` del `config.json`.

- **Resultado esperado:** La prueba generada es relevante y sigue el formato esperado.
- **Resultado Obtenido:**

[illegible]

[\[Ver imagen\]](#)

Resumen de Ejecución de Pruebas

- Se ejecutó correctamente: Si
- Género advertencia: No

Recomendar candidatos (Similitud de coseno):

Pruebas de Caja Negra (Funcionales)

Prueban el comportamiento del sistema desde la perspectiva del usuario.

Ejemplo de Caso de Prueba de Caja Negra:

Caso de Prueba: Validación de Recomendaciones según Preferencias de Usuario

- **Descripción:** Verificar que el sistema genere recomendaciones relevantes según las preferencias del usuario.
- **Precondiciones:** Archivo `interests_{project}.json` cargado con las preferencias.
- **Datos de Entrada:** Preferencias configuradas en el campo `columna_prompt` del archivo `config.json`.

Datos de Entrada:

```
src > service > cosine_similarity > 054 > {} config.json > ...
1  {
2      "columna_prompt": {
3          "IntFreeExperience": [
4              "12"
5          ]
6      },
7      "columnas_outputs": [
8          "intFreeName"
9      ],
10     "filtro_colaborativo": [
11         {
12             "columna": "intJobRequiredSkill",
13             "valor": "JavaScript"
14         },
15         {
16             "columna": "intUserPreferenceParamters",
17             "valor": "Creatividad"
18         }
19     ],
20     "umbral_similitud": 0.2,
21     "n_token": 5,
22     "json_file": "interest_054.json"
23 }
```

[\[Ver imagen\]](#)

- **Pasos para la Ejecución:**
 1. Configurar preferencias en el archivo **config.json**.
 2. Verificar las recomendaciones generadas.
- **Resultado esperado:** Las recomendaciones son relevantes y siguen el formato esperado.
- **Resultado Obtenido:**

```

Marcos [SSH: server_ia_test]
config.json .../045_2 {} config.json .../cosine_similarity M x
TecnicasML > src > service > cosine_similarity > {} config.json > project
1  {
2      "project": "054"
3  }

CHAT TERMINAL
zsh - cosine_similarity + v
•> cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s)
    in 0.03s
    Running `target/debug/cosine_similarity`
[{"intFreeName": "Brett Gonzalez", {"intFreeName": "Brenda Fa
rrell"}, {"intFreeName": "David Spears"}, {"intFreeName": "Mat
thew Robinson MD"}, {"intFreeName": "Lindsay Baker"}]

~/M/TecnicasML/s/s/cosine_similarity dev >3 !2 ?2
o >

```

[\[Ver imagen\]](#)

- **Observaciones:** Ninguna.

Resumen de Ejecución de Pruebas

- Se ejecutó correctamente: Si
- Género advertencia: No

Clasificación automática y eficiente para el candidato para el puesto (Neural Network User Segmentation):

Pruebas de Caja Negra (Funcionales)

Prueban el comportamiento del sistema desde la perspectiva del usuario.

Ejemplo de Caso de Prueba de Caja Negra:

- **Nombre del Caso de Prueba:** Validación de la Clasificación según Preferencias de Usuario
- **Descripción:** Verificar que el sistema genere clasificaciones relevantes según las preferencias del usuario.
- **Precondiciones:** Archivo `interests_{project}.json` cargado con preferencias.

Datos de Entrada:


```

1  {
2      "batch_size": 32,
3      "epochs": 15,
4      "encoding_dim": 3,
5      "json_file": "interest_054.json",
6      "model_save_path": "model_recruitment_054.h5",
7      "num_cluster": 3,
8      "random_seed": 42,
9      "categorical_columns": ["intUserPreferenceParameters"],
10     "exclude_columns": ["intFreeName", "intJobRequiredSkill"],
11     "segment_names": {
12         "0": ["Prioridad Alta", "High Priority", "alta"],
13         "1": ["Prioridad Media", "Medium Priority", "media"],
14         "2": ["Prioridad Baja", "Low Priority", "baja"]
15     },
16     "metrics": ["IntSkiEvaluationPunctuation", "IntFreeExperience"],
17     "new_user_data": {
18         "intUserPreferenceParameters": ["Liderazgo"],
19         "IntSkiEvaluationPunctuation": [8],
20         "IntFreeExperience": [10]
21     }
22 }

```

[\[Ver imagen\]](#)

- Pasos para la Ejecución:
 1. Ingresar preferencias en el campo columna_prompt del **config.json**.
 2. Ejecutar el sistema de clasificaciones mediante el **main.py**.
 3. Verificar las clasificaciones generadas.
- Resultado esperado: Las clasificaciones son relevantes y siguen el formato esperado.
- Resultado Obtenido:

```

94/94 0s 747us/step
/home/ia/Marcos/.venv/lib/python3.11/site-packages/sklearn/cluster/_kmeans.py:1416: FutureWarning: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
1/1 0s 14ms/step
Chosen segment names: {0: 'High Priority', 2: 'Prioridad Media', 1: 'Prioridad Baja'}
Segment name: High Priority
El nuevo usuario pertenece al High Priority.

~/M/TecnicasML/p/neural_network_user_segmentation dev >5 !1 ?1
> 

```

Ctrl+K to generate a command

Ln 3, Col 2 Tab Size: 4 UTF-8 LF {} JSON Cursor Tab

[\[Ver imagen\]](#)

- **Observaciones:** Ninguna.

Resumen de Ejecución de Pruebas

- Se ejecutó correctamente: Si
- Género advertencia: No

- (Redes Neuronales GPT Fine Tuned):

Pruebas de Caja Blanca (Unitarias)

Prueban el comportamiento interno del código fuente.

Caso de Prueba (**test_config_json**): Validación de la Estructura de Configuración

- **Descripción:** Asegura que el archivo **config.json** contiene las claves requeridas con los tipos correctos.
- **Precondiciones:** Archivo **config.json** cargado.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar **config.json**.
 2. Ejecutar la función **test_config_json()**.
- **Resultado Esperado:** El archivo contiene las claves y tipos correctos.
- **Resultado Obtenido:** Pasó
- **Observaciones:** Ninguna.

Caso de Prueba (**test_json_file_content**): Validación de la Estructura del archivo JSON de datos

- **Descripción:** Prueba para verificar si el archivo JSON especificado en la configuración existe y tiene la estructura correcta
- **Precondiciones:** Archivo **interest.json** cargado.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar **interest.json**.
 2. Ejecutar la función **test_json_file_content()**.
- **Resultado Esperado:** El archivo contiene las claves y tipos correctos.
- **Resultado Obtenido:** Pasó
- **Observaciones:** Ninguna.

Caso de Prueba (**test_algorithm_output**): Validación de la Estructura del output del algoritmo

- **Descripción:** Prueba para verificar si la salida del algoritmo es JSON válida y tiene la estructura correcta.
- **Precondiciones:** Archivos **config.json** e **interest.json** cargados.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar **config.json**.
 2. Cargar **interest.json**.
 3. Ejecutar la función **test_algorithm_output()**.
- **Resultado Esperado:** JSON válido.
- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Caso de Prueba (**test_prompt_generation**): Verificar generación de prompt

- **Descripción:** Prueba para verificar que la función **test_prompt_generation** genera correctamente el prompt desde el contenido del mensaje.
- **Precondiciones:** Archivo **config.json** cargado mediante **load_test_config()**. Carpeta de proyecto **test_project** existente.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar la configuración utilizando la función **load_test_config()**.
 2. Ejecutar la función **test_prompt_generation()**.
- **Resultado Esperado:** La función devuelve un prompt que sea una cadena o una lista.
- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Caso de Prueba (**test_model_loading**): Verificar carga del modelo

- **Descripción:** Prueba para verificar que la función **test_model_loading** carga correctamente el modelo y crea el pipeline.
- **Precondiciones:** Archivo **config.json** cargado mediante **load_test_config()**.
- **Datos de Entrada:** No aplica.
 1. **Pasos para la Ejecución:**
 2. Cargar la configuración utilizando la función **load_test_config()**
 3. Ejecutar la función **test_model_loading()**.
- **Resultado esperado:** El modelo y el pipeline son cargados correctamente.
- **Resultado Obtenido:** Pasó
- **Observaciones:** Ninguna.

Caso de Prueba (**test_whisper_config**): Verificar configuración del modelo Whisper

- **Descripción:** Prueba para verificar que la configuración del modelo Whisper sea correcta.
- **Precondiciones:** Archivo **config.json** cargado mediante **load_test_config()**.
- **Datos de Entrada:** No aplica.
 1. **Pasos para la Ejecución:**
 2. **Cargar la configuración utilizando la función **load_test_config()****
 3. **Verificar que la clave "whisper_model" exista en el archivo de configuración.**
 4. **Ejecutar la función **test_whisper_config()**.**
- **Resultado Esperado:** La clave "whisper_model" está presente en la configuración.
- **Resultado Obtenido:** Pasó
- **Observaciones:** Ninguna.

Caso de Prueba (**test_audio_transcription**): Verificar funcionalidad de transcripción de audio

- **Descripción:** Prueba para verificar que la función **test_audio_transcription** realiza correctamente la transcripción de un archivo de audio.
- **Precondiciones:** Archivo **config.json** cargado mediante **load_test_config()**.
- **Archivo de audio "test.mp3" disponible en la carpeta del proyecto.**
- **Datos de Entrada:** Archivo de audio: "test.mp3".
- **Pasos para la Ejecución:**
 1. **Cargar la configuración utilizando la función **load_test_config()**.**
 2. **Asegurarse de que el archivo "test.mp3" existe en la ubicación esperada.**
 3. **Ejecutar la función **test_audio_transcription()**.**
- **Resultado Esperado:** La función genera la transcripción del archivo "test.mp3" y la devuelve como una cadena.
- **Resultado Obtenido:** Pasó
- **Observaciones:** Ninguna.

Resultado de PYTEST

```
> pytest test_nlp_phi_whisper.py
===== test session starts =====
platform linux -- Python 3.11.10, pytest-8.3.4, pluggy-1.5.0
rootdir: /home/admin/development/Marcos/dazlabs_2024_dazlabs_nlp_whisper
collected 6 items

test_nlp_phi_whisper.py .....S [100%]

===== warnings summary =====
venv/lib/python3.11/site-packages/accelerate/utils/other.py:220
/home/admin/development/Marcos/dazlabs_2024_dazlabs_nlp_whisper/venv/lib/python3.11/site-packages/accelerate/utils/other.py:220: DeprecationWarning
: numpy.core is deprecated and has been renamed to numpy._core. The numpy._core namespace contains private NumPy internals and its use is discouraged
, as NumPy internals can change without warning in any release. In practice, most real-world usage of numpy.core is to access functionality in the pu
blic NumPy API. If that is the case, use the public NumPy API. If not, you are using NumPy internals. If you would still like to access an internal a
ttribute, use numpy._core.multiarray.
np.core.multiarray._reconstruct,
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 5 passed, 1 skipped, 1 warning in 6.34s =====
```

[\[Ver imagen\]](#)

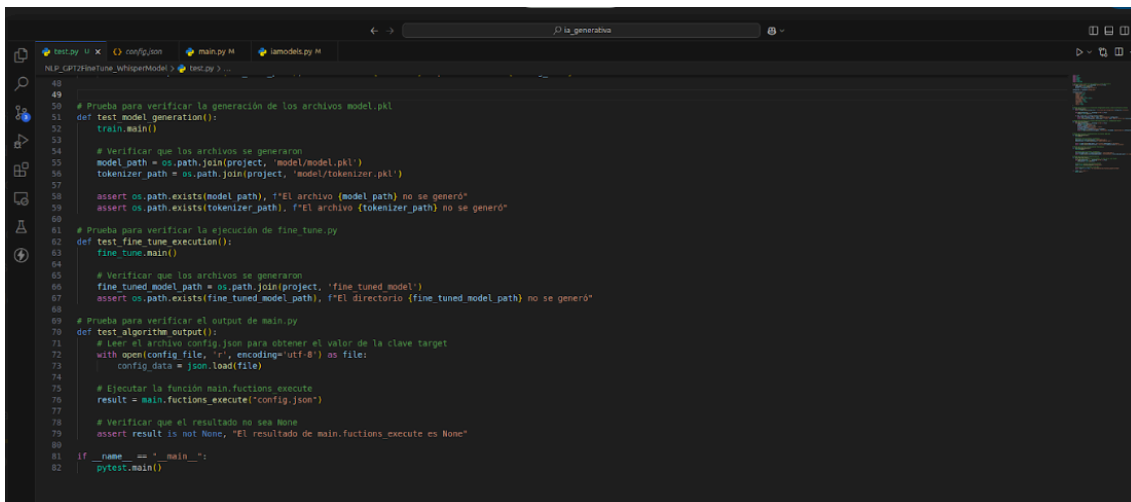
Resumen de Ejecución de Pruebas

- **Total de Casos de Prueba: 5**
- **Casos Pasados: 5**
- **Casos Fallados: 0**

Código del test:

```
1 import json
2 import os
3 import pytest
4 import main
5 import train
6 import fine_tune
7
8 # Leer el archivo config.json para obtener el número de proyecto
9 with open('config.json', 'r', encoding='utf-8') as file:
10     config_data = json.load(file)
11     project = config_data.get('project')
12
13 config_file = f'{project}/config.json'
14
15 config_structure = {
16     'model_name': str,
17     'dataset': list,
18     'prompt': str,
19     'prompt_type': str,
20     'whisper_model': (str, bool),
21     'is_screen': bool,
22     'max_length': int,
23     'temperature': float,
24     'top_k': int,
25     'top_p': float
26 }
27
28 # Prueba para verificar si el archivo de configuración existe y tiene la estructura correcta
29 def test_config_json():
30     assert os.path.exists(config_file), f'El archivo de configuración (config_file) no existe'
31
32     with open(config_file, 'r', encoding='utf-8') as file:
33         config = json.load(file)
34
35     for key, value_type in config_structure.items():
36         assert key in config, f'Clave {key} no encontrada en (config_file)'
37         assert isinstance(config[key], value_type), f'Clave {key} debería ser de tipo {value_type.__name__} en (config_file)'
38
39 # Prueba para verificar si el archivo CSV especificado en la configuración existe
40 def test_csv_file_content():
41     with open(config_file, 'r', encoding='utf-8') as file:
42         config = json.load(file)
43         dataset = config['dataset'][0]
44         assert dataset['type'] in ['csv', 'json']
45         filename = dataset['filename']
46         csv_file_path = f'{project}/dataset/{filename}'
47         assert os.path.exists(csv_file_path), f'El archivo {filename} especificado en (config_file) no existe'
```

[\[Ver imagen\]](#)

A screenshot of a code editor with a dark theme. The editor shows a Python file with test functions. The code includes comments in Spanish and uses standard Python testing libraries like unittest and os. The functions being tested are 'test_model_generation', 'test_fine_tune_execution', and 'test_algorithm_output'. The code is well-commented, explaining the purpose of each test and the assertions used. The file name at the top is 'la_generativa'.

[\[Ver imagen\]](#)

- (Algoritmo de Similitud de Coseno)

Pruebas de Caja Blanca (Unitarias)

Prueban el comportamiento interno del código fuente.

Caso de Prueba (**test_config_json**): Validación de la Estructura de Configuración

- **Descripción:** Validación de la estructura del archivo **config.json**.
- **Precondiciones:** Archivo **config.json** cargado en el proyecto.
- **Datos de Entrada:** N/A.
- **Pasos para la Ejecución:**
 1. Cargar el archivo **config.json**.
 2. Ejecutar la función **test_config_json()**.
- **Resultado esperado:** El archivo contiene todas las claves requeridas y tipos correctos.
- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Caso de Prueba: **test_json_file_content**

- **Descripción:** Verificación de la estructura del archivo JSON de datos especificados.
- **Precondiciones:** Archivo **interest.json** cargado.
- **Datos de Entrada:** N/A.
- **Pasos para la Ejecución:**
 1. Cargar el archivo **interest.json**.
 2. Ejecutar la función **test_json_file_content()**.
- **Resultado Esperado:** El archivo contiene las claves requeridas en los datos.

- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Caso de Prueba: **test_algorithm_output**

- **Descripción:** Validar la estructura de la salida generada por el algoritmo.
- **Precondiciones:** Archivos **config.json** e **interest.json** cargados.
- **Datos de Entrada:** N/A.
- **Pasos para la Ejecución:**
 1. Cargar el archivo de configuración y los datos.
 2. Ejecutar la función **test_algorithm_output()**.
- **Resultado Esperado:** JSON válido con recomendaciones relevantes.
- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Caso de Prueba: **test_algorithm_precision**

- **Descripción:** Validar que la precisión del algoritmo supera el umbral configurado.
- **Precondiciones:** Archivos **config.json** e **interest.json** cargados.
- **Datos de Entrada:** N/A.
- **Pasos para la Ejecución:**
 1. Cargar el archivo de configuración y los datos.
 2. Ejecutar la función **test_algorithm_precision()**.
- **Resultado esperado:** Precisión mayor o igual al umbral configurado.
- **Resultado Obtenido:** Precisión alcanzada: **85%**. Supera el umbral esperado de **80%**.
- **Observaciones:** Ninguna.

Caso de Prueba: **(test_main_algorithm_execution)**

- **Descripción:** Validar que el algoritmo ejecuta correctamente todas las fases, desde la carga de configuración hasta la salida final.
- **Precondiciones:** Archivos de configuración y datos correctamente configurados.
- **Pasos para la Ejecución:**
 1. Ejecutar el comando **cargo run**.
 2. Validar que la ejecución no produce errores.
 3. Comprobar que la salida contiene datos válidos.
- **Resultado Esperado:** Ejecución sin errores y salida con datos relevantes.
- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Resultado de PYTEST

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

    Running tests/cosine_similarity_test.rs (target/debug/deps/cosine_similarity_test-b5c6e3f9fb9a0ce0)

running 5 tests
Iniciando prueba test_main_algorithm_execution
Directorio de trabajo actual: "/home/kerly/Desktop/ML/TecnicasML/src/service/cosine_similarity"
test test_config_json ... ok
test test_algorithm_output ... ok
test test_json_file_content ... ok
test test_algorithm_precision ... ok
Salida del algoritmo: [{"intFreeName": "Brett Gonzalez"}, {"intFreeName": "Brenda Farrell"}, {"intFreeName": "David Spears"}, {"intFreeName": "Matthew Robinson MD"}, {"intFreeName": "Lindsay Baker"}]
test test_main_algorithm_execution ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.12s

    Doc-tests cosine_similarity

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

[\[Ver imagen\]](#)

Resumen de Ejecución de Pruebas

- **Total de Casos de Prueba: 5**
- **Casos Pasados: 0**
- **Casos Fallados: 0**

Código del test:

```
1 // tests/cosine_similarity_test.rs
2 use std::env;
3 use serde_json::{from_reader, Value};
4 use std::fs::File;
5 use std::collections::HashMap;
6 use cosine_similarity::{read_config, read_data, read_main_config, filter_data, generate_tf_idf_map, calculate_similarities, get_recommendations, select_top_recommendations};
7 use std::process::Command;
8
9 #[test]
10 fn test_config_json() {
11     let main_config = read_main_config("config.json").expect("No se pudo leer el archivo config.json");
12     let config_path = format!("{}/config.json", main_config.project);
13     let config = read_config(&config_path).expect("No se pudo leer el archivo de configuración específico");
14
15     // Validar claves en la configuración
16     assert!(!config.json_file.is_empty(), "La clave 'json_file' no debe estar vacía");
17     assert!(config.n_token > 0, "n_token debe ser mayor que 0");
18     assert!(((0.0..=1.0).contains(&config.umbrales_similitud), "umbrales_similitud debe estar entre 0.0 y 1.0");
19     assert!(!config.columnas_outputs.is_empty(), "La clave 'columnas_outputs' no debe estar vacía");
20     assert!(!config.filtro_colaborativo.is_empty(), "La clave 'filtro_colaborativo' no debe estar vacía");
21 }
22
23 #[test]
24 fn test_json_file_content() {
25     let main_config = read_main_config("config.json").expect("No se pudo leer el archivo config.json");
26     let config_path = format!("{}/config.json", main_config.project);
27     let config = read_config(&config_path).expect("No se pudo leer el archivo de configuración específico");
28     let data_path = format!("{}/{}/", main_config.project, config.json_file);
```

[\[Ver imagen\]](#)


```

29     let data = read_data(&data_path).expect("No se pudo leer el archivo de datos JSON");
30
31     // Validar la estructura de los datos
32     for item in &data {
33         for key in &config.columnas_outputs {
34             assert!(item.contains_key(key), "La clave '{}' no se encuentra en los datos", key);
35         }
36         for key in config.columna_prompt.keys() {
37             assert!(item.contains_key(key), "La clave '{}' no se encuentra en los datos", key);
38         }
39     }
40 }
41
42 #[test]
43 fn test_algorithm_output() {
44     let main_config = read_main_config("config.json").expect("No se pudo leer el archivo config.json");
45     let config_path = format!("{}/config.json", main_config.proyect);
46     let config = read_config(&config_path).expect("No se pudo leer el archivo de configuración específico");
47     let data_path = format!("{}/{}", main_config.proyect, config.json_file);
48     let data = read_data(&data_path).expect("No se pudo leer el archivo de datos JSON");
49
50     let filtered_data = filter_data(&data, &config);
51     let tf_idf_map = generate_tf_idf_map(&filtered_data, &config);
52     let similarities = calculate_similarities(&filtered_data, &tf_idf_map);
53     let recommendations = get_recommendations(filtered_data, similarities, &config);
54     let top_recommendations = select_top_recommendations(recommendations, &config);
55
56     // Validar la salida
57     assert!(!top_recommendations.is_empty(), "Las recomendaciones no deben estar vacías");

```

[\[Ver imagen\]](#)

```

58     for recommendation in &top_recommendations {
59         for key in &config.columnas_outputs {
60             assert!(recommendation.contains_key(key), "La clave '{}' no se encuentra en la recomendación", key);
61         }
62     }
63 }
64
65 #[test]
66 fn test_algorithm_precision() {
67     let main_config = read_main_config("config.json").expect("No se pudo leer el archivo config.json");
68     let config_path = format!("{}/config.json", main_config.proyect);
69     let config = read_config(&config_path).expect("No se pudo leer el archivo de configuración específico");
70     let data_path = format!("{}/{}", main_config.proyect, config.json_file);
71     let data = read_data(&data_path).expect("No se pudo leer el archivo de datos JSON");
72
73     let filtered_data = filter_data(&data, &config);
74     let tf_idf_map = generate_tf_idf_map(&filtered_data, &config);
75     let similarities = calculate_similarities(&filtered_data, &tf_idf_map);
76
77     // Calcular la precisión
78     let umbral_similitud = config.umbral_similitud;
79     let precision: f64 = similarities.iter().filter(|&score| score > umbral_similitud).count() as f64 / similarities.len() as f64;
80
81     let expected_precision = 0.8; // Precisión esperada de ejemplo
82     assert!(precision >= expected_precision, "La precisión {} está por debajo del umbral esperado {}", precision, expected_precision);
83 }
84
85 #[test]
86 fn test_main_algorithm_execution() {
87     println!("Iniciando prueba test_main_algorithm_execution");

```

[\[Ver imagen\]](#)

```

85  #[test]
86  fn test_main_algorithm_execution() {
87      println!("Iniciando prueba test_main_algorithm_execution");
88
89      // Imprimir el directorio actual
90      let current_dir = env::current_dir().expect("No se pudo obtener el directorio actual");
91      println!("Directorio de trabajo actual: {:?}", current_dir);
92
93      let output = Command::new("cargo")
94          .arg("run")
95          .output()
96          .expect("Fallo al ejecutar el comando cargo run");
97
98      if !output.status.success() {
99          println!("stdout: {}", String::from_utf8_lossy(&output.stdout));
100         println!("stderr: {}", String::from_utf8_lossy(&output.stderr));
101         panic!("El algoritmo principal no se ejecutó correctamente");
102     }
103
104     assert!(!output.stdout.is_empty(), "La salida del algoritmo no debe estar vacía");
105     println!("Salida del algoritmo: {}", String::from_utf8_lossy(&output.stdout));
106 }

```

[\[Ver imagen\]](#)

- (Neural Network User Segmentation)

Pruebas de Caja Blanca (Unitarias)

Prueban el comportamiento interno del código fuente.

Caso de Prueba (**test_config_json**): Validación de la Estructura de Configuración

- **Descripción:** Asegura que el archivo **config.json** contiene las claves requeridas con los tipos correctos.
- **Precondiciones:** Archivo **config.json** cargado.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar **config.json**.
 2. Ejecutar la función **test_config_json()**.
- **Resultado Esperado:** El archivo contiene las claves y tipos correctos.
- **Resultado Obtenido:** Pasó
- **Observaciones:** Ninguna.

Caso de Prueba (**test_json_file_content**): Validación de la Estructura del archivo JSON de datos

- **Descripción:** Prueba para verificar si el archivo JSON especificado en la configuración existe y tiene la estructura correcta
- **Precondiciones:** Archivo `interest.json` cargado.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar `interest.json`.
 2. Ejecutar la función `test_json_file_content()`.
- **Resultado Esperado:** El archivo contiene las claves y tipos correctos.
- **Resultado Obtenido:** **Falló.**
- **Observaciones:** Ninguna.

Caso de Prueba (`test_algorithm_output`): Validación de la Estructura del output del algoritmo

- **Descripción:** Prueba para verificar si la salida del algoritmo es JSON válida y tiene la estructura correcta.
- **Precondiciones:** Archivos `config.json` e `interest.json` cargados.
- **Datos de Entrada:** No aplica.
- **Pasos para la Ejecución:**
 1. Cargar `config.json`.
 2. Cargar `interest.json`.
 3. Ejecutar la función `test_algorithm_output()`.
- **Resultado Esperado:** JSON válido.
- **Resultado Obtenido:** Pasó.
- **Observaciones:** Ninguna.

Resultado de PYTEST

```
kerly@kerly-HP-ENVY-x360-2-in-1-Laptop-15-ey0xxx:~/Desktop/ML/TecnicasML/python_src/neural_network_user_segmentation$ pytest py_test
.py
===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.2.2, pluggy-1.5.0
rootdir: /home/kerly/Desktop/ML/TecnicasML/python_src/neural_network_user_segmentation
plugins: Faker-25.9.1
collected 3 items

py_test.py ... [100%]

===== 3 passed in 4.31s =====
kerly@kerly-HP-ENVY-x360-2-in-1-Laptop-15-ey0xxx:~/Desktop/ML/TecnicasML/python_src/neural_network_user_segmentation$
```

[\[Ver imagen\]](#)

Resumen de Ejecución de Pruebas

- **Total de Casos de Prueba:** 3
- **Casos Pasados:** 3
- **Casos Fallados:** 0

Código del test:

```
config.json .../recommendation_v2  py_test.py M X  config.json .../112  main.py M
python_src > neural_network_user_segmentation > py_test.py > ...
1 | import json
2 | import os
3 | import pytest
4 | import main
5 |
6 | # Leer el archivo config.json-1 para obtener el número de proyecto
7 | with open('config.json', 'r', encoding='utf-8') as file:
8 |     config_data = json.load(file)
9 |     project = config_data.get('project')
10 |
11 | config_file = f'{project}/config.json'
12 |
13 | config_structure = {
14 |     'batch_size': int,
15 |     'epochs': int,
16 |     'encoding_dim': int,
17 |     'json_file': str,
18 |     'model_save_path': str,
19 |     'num_cluster': int,
20 |     'random_seed': int,
21 |     'categorical_columns': list,
22 |     'exclude_columns': list,
23 |     'segment_names': dict,
24 |     'metrics': list,
25 |     'new_user_data': dict
26 | }
27 |
28 | def get_output_structure(config_file):
29 |     with open(config_file, 'r', encoding='utf-8') as file:
30 |         config = json.load(file)
31 |         output_structure = {col: str for col in config['categorical_columns']}
32 |         return output_structure
33 |
34 | output_structure = get_output_structure(config_file)
35 |
```

[\[Ver imagen\]](#)

```
35 |
36 | # Prueba para verificar si el archivo de configuración existe y tiene la estructura correcta
37 | def test_config_json():
38 |     assert os.path.exists(config_file), f"El archivo de configuración {config_file} no existe"
39 |
40 |     with open(config_file, 'r', encoding='utf-8') as file:
41 |         config = json.load(file)
42 |
43 |     for key, value_type in config_structure.items():
44 |         assert key in config, f"Clave '{key}' no encontrada en {config_file}"
45 |         assert isinstance(config[key], value_type), f"Clave '{key}' debería ser de tipo {value_type.__name__} en {config_file}"
46 |
47 | # Prueba para verificar si el archivo JSON especificado en la configuración existe y tiene la estructura correcta
48 | def test_json_file_content():
49 |     with open(config_file, 'r', encoding='utf-8') as file:
50 |         config = json.load(file)
51 |
52 |     json_file_path = f'{project}/{config["json_file"]}'
53 |     assert os.path.exists(json_file_path), f"El archivo {config['json_file']} especificado en {config_file} no existe"
54 |
55 |     with open(json_file_path, 'r', encoding='utf-8') as json_file:
56 |         data = json.load(json_file)
57 |
58 |     for item in data:
59 |         for key in config['categorical_columns']:
60 |             assert key in item, f"Clave '{key}' no encontrada en el elemento de datos en {json_file_path}"
61 |         for key in config['exclude_columns']:
62 |             assert key in item, f"Clave '{key}' no encontrada en el elemento de datos en {json_file_path}"
63 |
```

[\[Ver imagen\]](#)

```

63
64 # Prueba para verificar el output de main.py
65 def test_algorithm_output():
66     # Leer el archivo config.json para obtener el valor de las claves predict, label y label_2
67     with open(config_file, 'r', encoding='utf-8') as file:
68         config_data = json.load(file)
69         segment_names = config_data.get('segment_names')
70
71     # Ejecutar la función main.main
72     result = main.fuctions_execute("config.json")
73
74     # Verificar que el resultado no sea None
75     assert result is not None, "El resultado de main.fuctions_execute es None"
76
77     # Verificar que la respuesta esté en segment_names
78     valid_responses = [name for names in segment_names.values() for name in names]
79     assert result in valid_responses, f"La respuesta '{result}' no está en las opciones válidas {valid_responses}"
80
81

```

[\[Ver imagen\]](#)